

AI-Assisted ASIC Design Automation

Project report: a change-request pipeline with diagnose-and-route

Target block: PULP apb_gpio · sky130 · built 2026-10-02

Prepared for: the client and the faculty supervisor

Evidence: every run-derived figure is read from the run's manifest by report/collect_final.py; nothing is typed from memory. Appendix B lists the runs.

1. Summary

In plain terms: a person writes a change request in English. The pipeline looks the request up in a knowledge base built from the design's own specification, code and tests, decides whether the specification already allows it, writes a test that proves the change is needed, edits the hardware description until that test and every older test pass, and then carries the result through synthesis, gate-level simulation, place-and-route and signoff. A person approves every tool run and every decision that changes what the chip should do.

The part that is new is **diagnose-and-route**: when a test fails, the pipeline first decides **why** — a bug in the code, or a specification that never said what is right — and sends the problem to the agent that can fix it, instead of always asking for another code change.

Stage	Status	Evidence
Spec → RTL → RTL verification	Working, agents real	pilot runs, Section 4
Diagnose-and-route	Working; grounded by two checks	benchmark + pilots, Section 3
Synthesis (Yosys, sky130 hd)	Working, content-hash cached	1478 cells
Gate-level verification	Working; every RTL test re-run on the netlist	14/14 on the agent-fixed design
Place-and-route + signoff (OpenROAD)	Working, 100 MHz	WNS +2.267 ns, 0 DRC, LEC equivalent
Change-request pipeline (the product)	Working for requests the spec supports; new features limited by the model	Section 5
Knowledge base (RAG)	Integrated in every agent; write-back of confirmed fixes	Section 7
Second design (non-PULP)	Tool chain runs from project.json alone	10/10 gate-level, signoff pass

Headline results

Diagnosis: with the quote-grounding checks, the hosted model labels 9/9 benchmark failures acceptably; the local 7B model without the checks labelled an unspecified behaviour as a code bug every time, citing a sentence that is not in the spec.

End to end: 2 pilot runs reach signoff with no person editing code; a supported change request goes from English to signoff in 44 s.

Honest limits: a brand-new feature (a new interrupt type) is not yet built by the agents end to end, and the free model tier's daily limit is the practical bottleneck. Section 9.

2. What was built

The client's direction (2 October): the pipeline is the product and the knowledge base is its centre; the client will replace the PULP data with its own. So nothing in the request path depends on this block. One file, `project.json`, names the design, its specification, its testbench and the folders that go into the knowledge base, and is editable from the web app.

2.1 Components

Component	What it does	Where
Driver	Runs the stages, enforces the 3-attempt cap from the manifest, asks for every approval	<code>orchestrator/driver.py</code>
Request flow	English request → analysis → spec decision → test → RTL change → full flow	<code>orchestrator/request_flow.py</code>
Analysis agent	Compares the request with the spec: agrees / extends / conflicts, with a verified quote	<code>agents/analysis_agent.py</code>
Test agent	Writes a new test in the testbench's own style; it must fail on the old design	<code>agents/test_agent.py</code>
Diagnosis	Classifies each failure and routes it (Section 3)	<code>agents/diagnose.py</code>
Spec agent	Drafts a clarification with both readings; a person picks	<code>agents/spec_agent.py</code>
RTL agent	Up to 3 candidate edits, each linted and tested before any reaches the design	<code>agents/rtl_agent.py</code>
Knowledge base	Hybrid vector + keyword retrieval over spec, RTL, testbench and confirmed fixes	<code>phase1_rag/rag/</code>
Back end	Yosys synthesis, gate-level tests, OpenROAD place-and-route and signoff	<code>flow/, tb/run_tests.py --gate</code>
Web app	New request, approvals, decisions, runs, knowledge, project settings	<code>app/server.py</code>
Skills	Edit rules for the RTL agent; procedure and test-writing rules for verification	<code>skills/*/SKILL.md</code>

2.2 Safety properties kept on purpose

- The retry cap is enforced by the driver reading `attempt_count`, never by a prompt.
- A person approves every Verilator, Yosys and OpenROAD run, including cache hits.
- A gate-level failure can never be called a spec question: the same spec passed one stage earlier.
- Gate-level fixes are made in RTL and re-synthesised; the netlist is never edited.
- Only confirmed fixes are written back to the knowledge base.
- The synthesis/signoff cache key covers RTL, tool versions, PDK and constraint files.

3. Diagnose-and-route

The old design's blind spot: it knew *that* a test failed, not *why*, so it always asked for another code change. If the specification never said what was right, that burned the whole retry budget patching code against a moving target.

Diagnosis	Meaning	Routed to
implementation_bug	The spec states the behaviour; the RTL does something else	RTL agent
spec_ambiguity	The spec does not state it, or two correct readings exist	Spec agent → person decides
unknown	Cannot tell from the evidence	Person

3.1 Grounding checks

A model asked "is this a bug?" tends to say yes. Two checks stop that. A claimed `implementation_bug` must quote the specification sentence that states the expected behaviour, and (1) the quote must exist in the specification and (2) a separate yes/no question must agree that it states the failing behaviour. If either fails, the failure is not treated as a code bug.

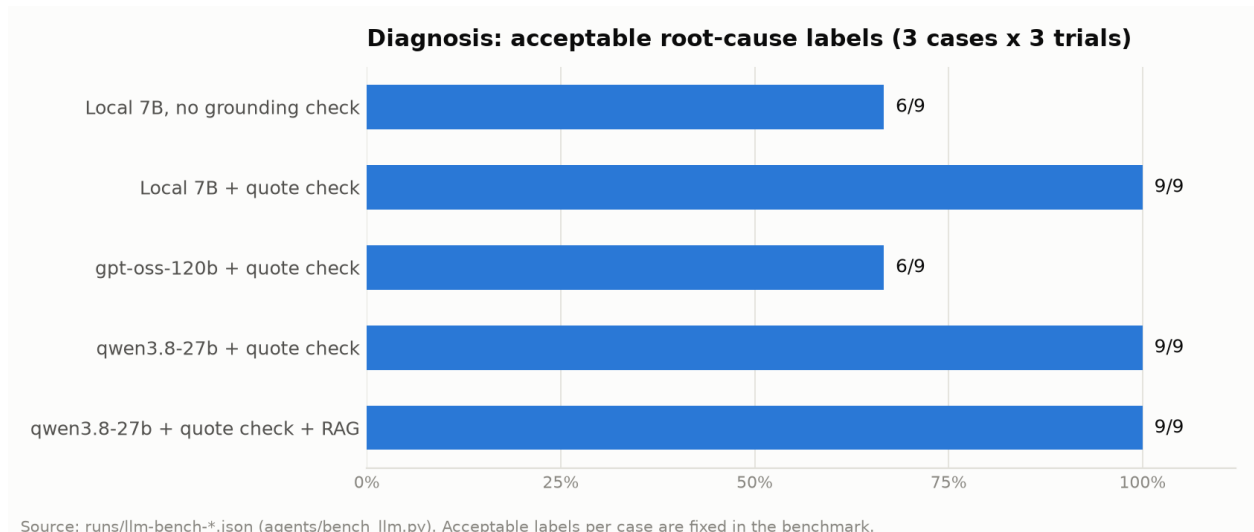


Figure 1. Diagnosis benchmark: three real failure cases, three trials each. The acceptable labels per case are fixed in `agents/bench_llm.py`; the interrupt-level case is genuinely arguable, so both labels are accepted.

gpt-oss-120b, the larger model, labelled the deliberately injected code bug as a spec question in 3 of 3 trials. It was not used for diagnosis.

3.2 Pilot runs

Each pilot is a complete run from one command, archived with its manifest. Runs 1–3 use the hosted model; the -v4 runs are the same cases on the local 7B model.

Run	Case	Routing	Outcome	Time
groq-pilot1-irq-level	Interrupt is a pulse; spec says level	spec_agent	signoff, no person needed	3.0 min
groq-pilot2-impl-bug	Injected rising-edge bug	rtl_agent	signoff, no person needed	35 s
groq-pilot3-spec-gap	Behaviour the spec calls 'RFU'	spec_agent → rtl_agent → rtl_agent	escalated to a person	88 s
pilot1-irq-level-v4	Interrupt level (local 7B)	spec_agent → rtl_agent → rtl_agent	signoff after a person's fix	7.2 min

Run	Case	Routing	Outcome	Time
pilot2-impl-bug-v4	Injected bug (local 7B)	rtl_agent → rtl_agent → rtl_agent	signoff after a person's fix	3.4 min
pilot3-spec-gap-v4	RFU gap (local 7B)	spec_agent → rtl_agent → rtl_agent	escalated to a person	4.3 min
pilot0-weak-test	Weak-test case (local 7B)	rtl_agent	signoff, no person needed	3.0 min

Routing was right on the first attempt in every pilot: the two specification gaps went to the spec agent, the injected bug went to the RTL agent. Where a run escalated, it was the code change that failed, not the routing: the local model could not write the fix, and the 'RFU' case asks for a feature that does not exist yet.

3.3 Against the old design

To measure what diagnose-and-route changes, the driver has a - -baseline mode that skips diagnosis and sends every failure to the RTL agent, as the old design did. Everything else (cap, approvals, screening) is identical.

Case	With diagnosis	Baseline (old design)
Interrupt is a pulse; spec says level	2 verification attempts; spec clarified by a person; signoff, no person needed	2 verification attempts; tests pass

What the comparison shows

On the interrupt-level case the old design reached a passing design in the same number of attempts: the test happened to encode the reading the RTL agent could implement. Diagnose-and-route did not save retries there.

What it changed is who decided. The baseline silently changed the chip's behaviour to match the test; diagnose-and-route stopped, showed both readings to a person, and wrote the chosen one into the specification, so the spec and the RTL agree afterwards and the next run starts from a settled spec.

The case where routing saves the retry budget outright is a specification gap the RTL agent cannot close by editing code (the 'RFU' case). Its baseline run is listed when the model quota allows it to complete.

4. The RTL agent

The RTL agent works by generate-and-test: up to three candidate edits at increasing temperature, each applied to a scratch copy, linted, and run against the failing test plus a smoke set behind one approval. The first candidate that passes is applied; edits that only change comments, are oversized, or break other tests are rejected before they reach the design.

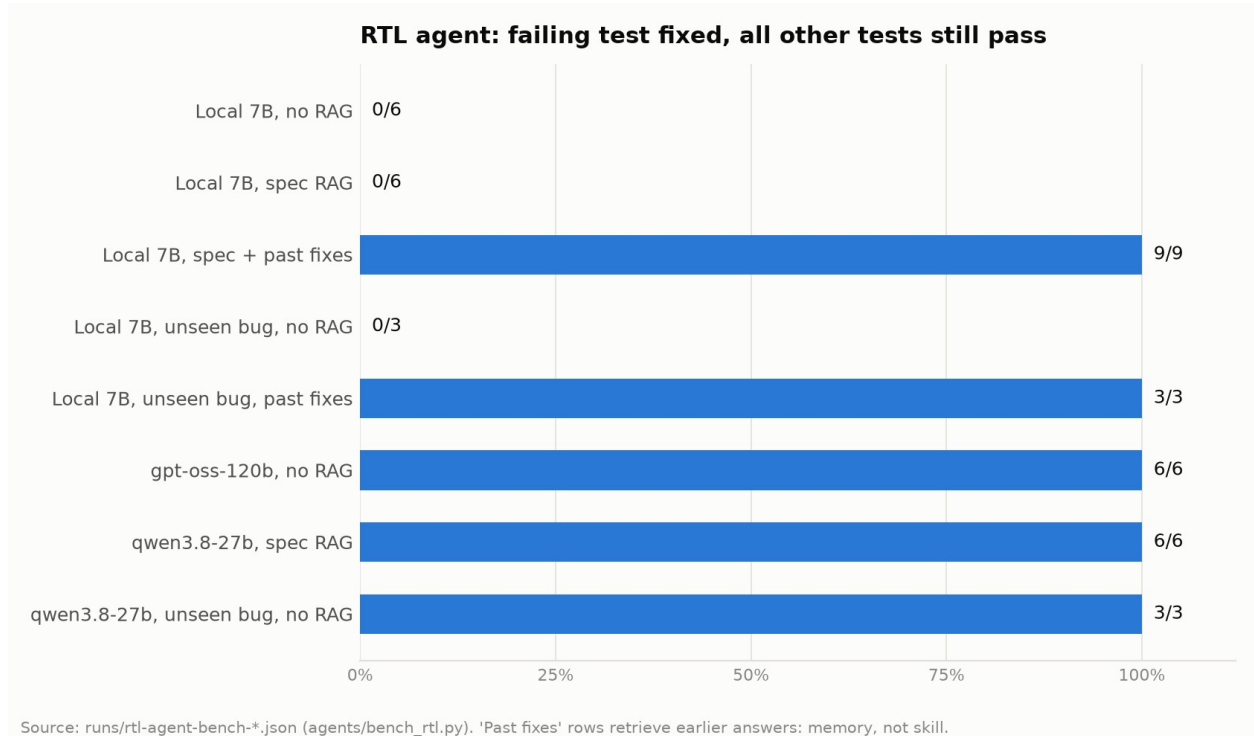


Figure 2. RTL repair benchmark. A trial counts only if the failing test passes and every other test still passes. The 'past fixes' rows retrieve earlier confirmed fixes from the knowledge base, so they measure memory, not skill; the 'unseen bug' rows are the fair comparison.

4.1 A weak test lets a wrong fix through

In `pilot0-weak-test` the agent's `fix interrupt = r_status[0]` passed all 14 tests and was wrong for pads 1–31, because no `interrupt` test used another pad. The test now also checks pad 5, the knowledge-base entry was marked retracted, and the correct fix is `assign interrupt = |r_status;`. The verification skill now requires checking more than one instance. This is the clearest example in the project of tests being the real specification.

5. Change-request pipeline

1. **Retrieve:** spec, RTL, testbench and past-change passages, balanced per kind.
2. **Compare:** verdict agrees / extends / conflicts, each claim backed by a spec quote that must exist.
3. **Spec decision:** if not 'agrees', a person decides whether the specification changes.
4. **Test first:** a new test in the testbench's style; it must fail on the unchanged design.
5. **RTL change:** located through the knowledge base; generate-and-test screening.
6. **Verify and sign off:** new test + full regression, synthesis, gate level, place-and-route.
7. **Write back:** the confirmed change is added to the knowledge base and the index rebuilt.

5.1 Results

Run	Verdict	Test	Candidates / applied	Outcome	Time
req-test1	agrees	irq_level_latched_status	2 / 1	signoff	44 s
req-test2	conflicts	inttype_level_high	9 / 0	escalated to a person	4.9 min
req-test2b	conflicts	inttype_level_high	3 / 0	escalated to a person	25.8 min
req-test2c	conflicts	—	0 / 0	escalated to a person	9.4 min
req-test2d	conflicts	inttype_reserved_level_high	8 / 0	escalated to a person; quota fallback	11.1 min
req-test2e	conflicts	gpio_intlevel_high_check	9 / 0	escalated to a person; quota fallback	10.6 min
req-test2f	conflicts	—	0 / 0	escalated to a person	57 s
req-test2g	conflicts	inttype_level_high	9 / 0	escalated to a person	5.7 min

req-test1 ("the interrupt output must stay asserted as a level until INTSTATUS is read") went from English to signoff with no person editing anything. Its signoff was a cache hit on an earlier pilot: the agent produced RTL with exactly the same content as the pilot's fix, so the content-hash key matched.

The req-test2* runs are one new-feature request (make INTTYPE = 11 a level-high interrupt) tried as the pipeline was improved. Retrieval, the 'conflicts' verdict, the spec decision and a correctly failing test worked every time. The code change did not land. Two causes were found and fixed:

- The agent-written test expected INTSTATUS to clear when the pin went low, but the specification says it clears when read. Correct RTL could never pass. The pipeline now reviews the test when every candidate fails the same check, and the test-writing rules forbid checks the specification does not state.
- Several attempts were cut short when the model's daily quota ran out and the run continued on the local 7B model, which cannot build new features.

6. Synthesis, gate level and signoff

Measure	Value
Synthesis (Yosys, sky130_fd_sc_hd tt)	1478 cells, 19,633 μm^2
Gate-level simulation	14/14 RTL tests on the netlist (cell models generated from the liberty file)
Clock	100 MHz (10 ns)
Setup worst slack	+2.267 ns
Hold worst slack	+0.499 ns
Routing DRC errors	0
Logic equivalence (netlist vs layout)	equivalent
Standard cells after PnR	2809
Die area	54,906 μm^2
Power (estimate)	5.444 mW
Place-and-route time	2.3 min

Source: runs/groq-pilot1-irq-level/manifest.json, the design with the agent's interrupt fix. The PDK's own gate models gave 5 false failures under Verilator (ignored delays collapse the synchroniser), so gate-level simulation uses cell models generated from the liberty file. Single corner (typical) only.

7. Knowledge base

Measure (12 validation queries, MiniLM)	Before fixes	Final
hit@1	17%	50%
hit@3	50%	58%
MRR	0.319	0.562

Source: `runs/m2-retrieval/retrieval_report.md` and `ANALYSIS.md`. The gains came from fixing three keyword-index defects and one chunker gap that only appeared on real PULP code. Meaning-based retrieval stays weak (0% hit@1 on paraphrased questions) because the specification is terse; larger embedding models were no better. The knowledge base now also indexes the testbench and every confirmed change, and every agent records which passages it used.

Gotcha kept in the design: vector search always returns its nearest neighbours, so an absent answer still looks confident. Agents treat retrieved passages as material to reason over, and quotes must be found verbatim in the specification.

8. A second design: swapping in the client's data

The client will replace the PULP block with its own. To test that before it matters, a second block was written for this project, deliberately unlike the first: `apb_timer`, a 32-bit periodic timer with a prescaler and a compare-match interrupt (`designs/apb_timer/`). It has its own specification, RTL and a 10-test testbench following the same contract. Then, exactly as a client would, `project.json` was pointed at it and the knowledge base rebuilt; no script was told the design's name.

Step	Result
Knowledge base	Rebuilt from the timer's spec, RTL and testbench only; <code>apb_gpio</code> 's past fixes excluded (they belong to another design)
Lint (Verilator -Wall)	PASS, no waivers needed
RTL tests	10/10. The counting tests' expected values (23, 5, 3) were derived by hand from the spec and the bus timing, not read off the RTL
Synthesis (Yosys, sky130 hd)	539 cells, 5,415 μm^2
Gate-level simulation	10/10
Place-and-route + signoff, 100 MHz	setup +3.433 ns, hold +0.529 ns, 0 DRC, LEC equivalent, 796 cells, 0.77 mW

Doing this found eleven places where the tool layer still assumed `apb_gpio`, all now read from `project.json` or from the testbench itself: the top-module name, the RTL path, the testbench module and default test list, the smoke set, lint waivers, the netlist file name, the place-and-route design name and summary paths, the clock port and period, and the knowledge-base export of past fixes. The clock moved into `project.json` (`design.clock`) and into the synthesis/signoff cache key, so changing it invalidates cached results. `apb_gpio` was re-run afterwards: same tests, signoff still passes.

Not yet shown on the second design: a change request through the agents, which needs model quota (Section 10).

9. Limitations and lessons

- **Model size decides RTL repair.** The local 7B model fixed 0 of 6 benchmark cases without retrieved past fixes; the hosted 27B model fixed 6 of 6. Diagnosis and routing work with the small model; code changes need the larger one.
- **Free-tier quota is the bottleneck.** Groq's free tier allows 200,000 tokens per model per rolling day, roughly 7–10 request runs. The pipeline falls back to the local model automatically and records it, but that model cannot build features. A client deployment needs a paid tier or another provider.
- **New features are the hard case.** Requests the specification supports work end to end; a new behaviour needs the test, the spec change and the RTL to agree, and every step depends on the model.
- **Tests are the real specification.** A weak test let a wrong fix pass (Section 4.1); a wrong test blocked a right fix (Section 5.1). Both are now checked.
- **Two blocks, one corner.** The agents were measured on `apb_gpio`; the second design has been through the tool chain but not yet through a change request. Signoff is at the typical corner only.
- **Engineering hygiene matters.** Simulator build folders were left behind on every run and filled the disk (~30 GB in a day); runs now delete them.

10. Remaining work

1. Complete the new-feature request and the 'RFU' baseline run when model quota allows, then rebuild this report (its tables fill in automatically).
2. Run a change request on the second design (apb_timer) when model quota allows; then on the client's own block.
3. Multi-corner signoff (slow and fast corners).
4. Team review of the M2 validation labels; decide the folder rename (the space in the path forces workarounds).
5. Put the project under version control and back it up.
6. Rehearse the final demo and record a backup video.

Appendix A — Reproducing

```
source tools/env.sh
./start_app.sh                                # web app on http://localhost:8080
./run_flow.sh --run-id X --auto-approve        # full flow on the upstream block
./run_flow.sh --run-id X --base-run groq-pilot1-irq-level --inject int_rise_bug
./run_flow.sh --run-id X --baseline --stop-after verify_rtl # old design
./run_flow.sh --run-id X --request-file request.txt        # change request
python3 report/collect_final.py && python3 report/plot_final.py
cd report && node build_final.js
```

Appendix B — Runs used

Run	Model	Final stage
groq-pilot1-irq-level	qwen/qwen3.8-27b	done
groq-pilot2-impl-bug	qwen/qwen3.8-27b	done
groq-pilot3-spec-gap	qwen/qwen3.8-27b	failed
pilot1-irq-level-v4	qwen2.5-coder-7b-local	done
pilot2-impl-bug-v4	qwen2.5-coder-7b-local	done
pilot3-spec-gap-v4	qwen2.5-coder-7b-local	failed
pilot0-weak-test	qwen2.5-coder-7b-local	done
baseline-pilot1-irq-level	qwen/qwen3.8-27b	done
req-test1	qwen/qwen3.8-27b	done
req-test2	qwen/qwen3.8-27b	failed
req-test2b	qwen/qwen3.8-27b	failed
req-test2c	qwen/qwen3.8-27b	failed
req-test2d	qwen/qwen3.8-27b (fell back -> qwen2.5-coder-7b-local)	failed
req-test2e	qwen/qwen3.8-27b (fell back -> qwen2.5-coder-7b-local)	failed
req-test2f	openai/gpt-oss-120b	failed
req-test2g	openai/gpt-oss-120b	failed